Axellio Xpress Platform RESTful API

Version: 5.1.0

Overview

The Axellio Xpress Platform provides its API via a RESTful web service interface, used for managing and controlling the system. This API provides comprehensive endpoints for packet capture extraction, flow analysis, file system operations, task management, analytics, and system configuration.

Security

API security is provided through two layers of protection:

- **SSL/TLS v1.2** provided by HTTPS
- **JWT token-based authentication** or **API key authentication**
  - Every request requires either a JWT token or API key in the authorization header
  - JWT tokens: `Authorization: Bearer <token>`
  - API keys: `Authorization: Bearer <api_key>`
  - Tokens contain user information for login status and permissions
  - API keys provide programmatic access with optional expiration

Authentication Methods

1. **Session-based (JWT)**: Login with username/password to receive a JWT token
2. **API Keys**: Create long-lived API keys for programmatic access

Quick Start

1. **Authentication**:
   - Obtain a JWT token via `/axapi/login` with username and password

    - Or create an API key via `/axapi/apikeys` for programmatic access

2. **Making Requests**:
    - Include token in header: `Authorization: Bearer <token>`
    - All endpoints use JSON for request/response payloads
    - Use HTTPS for all connections
3. **Error Handling**:
    - `401 Unauthorized`: Missing, bad, or expired token/key
    - `403 Forbidden`: Insufficient permissions
    - `400 Bad Request`: Invalid payload or parameters
    - `500 Internal Server Error`: Server-side error

API Base URL

```
https://<server>/axapi
```

Response Format

All API responses follow a consistent JSON format with appropriate HTTP status codes.

Success Response Example

```
{
  "data": { ... }
}
```

Error Response Example


```
{
  "code": 401,
  "message": "Unauthorized",
  "errors": [{
    "domain": "Authorization",
    "reason": "Invalid Token",
    "message": "Token has expired",
    "instance": null
  }]
}
```

Authentication

API Login (Get Token)

Request a login token using username and password

URI: `/axapi/login`

Method: `POST`

Headers:

- `Content-Type: application/json`

Request Payload:

```
{"username": string,"password": string}
```

- [required] Username for the user
- [required] Password for the user

Response:

Success (200):

```
{"token": string}
```

Error:

```
{"message": string}
```

Status Codes:

- `201` - - OK
- `400` - Bad request, something invalid/missing in the payload
- `401` - Username or password does not match

API Refresh (Refresh Token)

Ensures that the token is refreshed if provided a valid username and currently active token

URI: `/axapi/refresh`

Method: `POST`

Headers:

- `Content-Type: application/json`

Request Payload: None

Response:

Success (200):

```
{"message": string}
```

Status Codes:

- `201` - - OK
- `401` - Username or token does not match

API Validate

Ensures validation and correct routing on initial app load

URI: `/axapi/validate`

Method: `POST`

Headers:

- `Content-Type: application/json`

Request Payload: None

Response:

Success (200):

```
{"redirectPage": string}
```

Status Codes:

- `201` - - OK
- `401` - Username or token does not match

API Logout

Request a login token using username and password

URI: `/axapi/logout`

Method: `POST`

Request Payload:

```
{"token": string}
```

- The token to be logged out

Response:

Success (200):

```
{"token": string}
```

Error:

```
{"message": string}
```

Status Codes:

- `200` - - OK
- `400` - Bad request, something invalid/missing in the payload
- `401` - Missing/invalid token provided

API Key Management

API Key Create

Create a new API key for the authenticated user

URI: `/axapi/apikeys`

Method: `POST`

Headers:

- `Authorization: Bearer <token>`
- `Content-Type: application/json`

Request Payload (JSON):

```
{
  "description": string,
  "expires": string
}
```

- `description` - [required] Description of API key purpose
- `expires` - [optional] Expiration (e.g., "90d", "1y")

Response:

Success (201):

```
{
  "key_id": integer,
  "api_key": string,
  "user": string,
  "description": string,
  "expires_days": integer,
  "message": string
}
```

- `key_id` - The ID of the created API key
- `api_key` - The newly created API key (shown only once)
- `user` - Username that owns this key
- `description` - Description of the API key
- `expires_days` - Number of days until expiration
- `message` - Success message

Error (400/401/500):

```
{
  "error": string
}
```

- `error` - Error description

Status Codes:

- `201` - Created successfully
- `400` - Invalid request payload
- `401` - Authentication required
- `500` - Internal server error

API Key List

List all API keys for the authenticated user

URI: `/axapi/apikeys`

Method: `GET`

Headers:

- `Authorization: Bearer <token>`

Request Payload: None

Query Parameters:

- `include_revoked` - [optional] Include revoked keys in results (default: false)

Response:

Success (200):

```
{
  "keys": [
    {
      "id": integer,
      "key_prefix": string,
      "user": string,
      "description": string,
      "created_at": integer,
      "expires_at": integer,
      "last_used_at": integer,
      "revoked": boolean
    }
  ],
  "count": integer
}
```

- `keys` - Array of API keys (prefixes only, not full keys)
  - `id` - Unique identifier for the key
  - `key_prefix` - Prefix of the API key (for identification)
  - `user` - Username that owns this key
  - `description` - Description of the API key

    - `created_at` - Unix timestamp when key was created
    - `expires_at` - Unix timestamp when key expires
    - `last_used_at` - Unix timestamp of last usage
    - `revoked` - Whether the key has been revoked
- `count` - Total number of keys returned

Error (401/500):

```
{
  "error": string
}
```

- `error` - Error description

Status Codes:

- `200` - Success
- `401` - Authentication required
- `500` - Internal server error

API Key Info

URI: `/axapi/apikeys/<key_id>` Get detailed information about a specific API key

Method: `GET`

Headers:

- `Authorization: Bearer <token>`

Request Payload: None (key_id is specified in the URI path)

Response:

Success (200):

```
{
  "id": integer,
  "key_prefix": string,
  "user": string,
  "description": string,
  "created_at": integer,
  "expires_at": integer,
  "last_used_at": integer,
  "revoked": boolean,
  "is_expired": boolean
}
```

- `id` - Unique identifier for the key
- `key_prefix` - Prefix of the API key
- `user` - Username that owns this key
- `description` - Description of the API key
- `created_at` - Unix timestamp when key was created
- `expires_at` - Unix timestamp when key expires
- `last_used_at` - Unix timestamp of last usage
- `revoked` - Whether the key has been revoked
- `is_expired` - Whether the key has expired

Error (401/403/404):

```
{
  "error": string
}
```

- `error` - Error description

Status Codes:

- `200` - Success
- `401` - Authentication required

- `403` - Access denied (not owner)
- `404` - Key not found

API Key Revoke

Revoke an API key (cannot be reversed)

URI: `/axapi/apikeys/<key_id>`

Method: `DELETE`

Headers:

- `Authorization: Bearer <token>`

Request Payload: None (key_id is specified in the URI path)

Response:

Success (200):

```
{
  "message": string,
  "key_id": integer
}
```

- `message` - Confirmation message
- `key_id` - ID of the revoked key

Error (401/403/404):

```
{
  "error": string
}
```

- `error` - Error description

Status Codes:

- `200` - Successfully revoked
- `401` - Authentication required

- `403` - Access denied (not owner)
- `404` - Key not found

Task Management

Task Management

Running Tasks

Returns list of running tasks and their information

URI: `/axapi/task`

Method: `GET`

Request Payload: None

Response:

Success (200):

```
{
  "data": {
  task_guid: {
  "directory": task file loc"info": {
  "bytes_sent": int,
  "comment": string,
  "downloaded": bool,
  "end_time": string,
  "filter": string,
  "flowpreview_bytes": int,
  "flowpreview_packets": int,
  "flowpreview_percent": int,
  "guid": str,
  "output_length": int,
  "percent": int,
  "pid": int,
  "start_time": string,
  "status": string,
  "streamIds": string,
  "time_started": string,
  "type": string,
  "user": string,
  "ctid": string
}
}
}
}
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided

Task Information

Returns task information based on GUID or CTID

URI: `/axapi/task/<guid>` Returns task based on GUID or CTID

Method: `GET`

Request Payload: None

Response:

Success (200):

```
{
  "data": {
  task_guid: {
  "directory": task file loc"info": {
  "bytes_sent": int,
  "comment": string,
  "downloaded": bool,
  "end_time": string,
  "filter": string,
  "flowpreview_bytes": int,
  "flowpreview_packets": int,
  "flowpreview_percent": int,
  "guid": str,
  "output_length": int,
  "percent": int,
  "pid": int,
  "start_time": string,
  "status": string,
  "streamIds": string,
  "time_started": string,
  "type": string,
  "user": string,
  "ctid": string
}
}
}
}
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided

Latest task

Returns most recently started task

URI: `/axapi/task/latest`

Method: `GET`

Request Payload: None

Response:

Success (200):

```
{
  "data": {
  task_guid: {
  "directory": task file loc"info": {
  "bytes_sent": int,
  "comment": string,
  "downloaded": bool,
  "end_time": string,
  "filter": string,
  "flowpreview_bytes": int,
  "flowpreview_packets": int,
  "flowpreview_percent": int,
  "guid": str,
  "output_length": int,
  "percent": int,
  "pid": int,
  "start_time": string,
  "status": string,
  "streamIds": string,
  "time_started": string,
  "type": string,
  "user": string,
  "ctid": string
}
}
}
}
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided

Abort Task

Abort task based on GUID or CTID

URI: `/axapi/task/<guid>`

Method: `DELETE`

Request Payload: None

Response:

Success (200):

```
{"guid": string}
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided

User Management

User Management

Users Retrieval

Returns all entries of users table in database

URI: `/axapi/users`

Method: `GET`

Headers:

- `Content-Type: application/json`

Request Payload: None

Response:

Success (200):

```
{
  "users": {
    "id": int,
    "username": string,
    "roles": Array<String>
  }
}
```

- `users` - User information object
  - `id` - The ID of user in the users table
  - `username` - The username of user in the users table
  - `roles` - The roles of user in the roles table

Status Codes:

- `201` - OK

- `401` - Token invalid
- `500` - Error

Roles Retrieval

URI: `/axapi/roles` Grabs all entries of roles table in database

Method: `GET`

Headers:

- `Content-Type: application/json`

Request Payload: None

Response:

Success (200):

```
{
  "roles": {
    "id": int,
    "name": string
  }
}
```

- `roles` - Role information object
  - `id` - The ID of the role in the roles table
  - `name` - The name of the role in the roles table

Status Codes:

- `201` - OK
- `401` - Token invalid
- `500` - Error

Groups Retrieval

Returns all entries of groups table in database

URI: `/axapi/groups`

Method: `GET`

Headers:

- `Content-Type: application/json`

Request Payload: None

Response:

Success (200):

```
{
  "groups": {
    "id": int,
    "name": string
  }
}
```

- `groups` - Group information object
  - `id` - The ID of the group in the groups table
  - `name` - The name of the group in the group table

Status Codes:

- `201` - OK
- `401` - Token invalid
- `500` - Error

Group Roles Retrieval

URI: `/axapi/group_roles` Grabs all entries of group roles table in database

Method: `GET`

Headers:

- `Content-Type: application/json`

Request Payload: None

Response:

Success (200):

```
{
  "group_roles": {
    "id": int,
    "name": string,
    "roles": Array<String>
  }
}
```

- `group_roles` - Group roles information object
  - `id` - The ID of group in the group table
  - `name` - The name of group in the group table
  - `roles` - The roles of group in the roles table

Status Codes:

- `201` - OK
- `401` - Token invalid
- `500` - Error

User Role Check

Check User's system groups and assigns roles accordingly

URI: `/axapi/user/role_check?<username>`

Method: `GET`

Headers:

- `Content-Type: application/json`

Request Payload: None

Response:

Success (200):

```
{
  "username": string,
  "roles": Array<String>
}
```

- `username` - The username
- `roles` - Array of roles assigned to said user's groups

Status Codes:

- `201` - OK
- `400` - Bad request, something invalid/missing in the payload
- `401` - Username or password does not match

User Add

Adds a user to the users table in the database

URI: `/axapi/user`

Method: `POST`

Headers:

- `Content-Type: application/json`

Request Payload (JSON):

```
{
  "username": string,
  "roles": Array<String>
}
```

- `username` - [required] Username for the user

- `roles` - Array of roles to assign user in user-roles table

Response:

Success (200):

```
{
  "success": boolean,
  "message": string
}
```

- `success` - Returns if user add was successful
- `message` - Message indicating if user was added or error contents

Status Codes:

- `201` - OK
- `400` - Bad request, something invalid/missing in the payload
- `401` - Username or password does not match

User Modify

Modifies a user in the users table in the database

URI: `/axapi/user`

Method: `PUT`

Headers:

- `Content-Type: application/json`

Request Payload (JSON):

```
{
  "username": string,
  "roles": {
    "toAdd": Array<String>,
    "toRemove": Array<String>
  }
}
```

- `username` - [required] Username for the user
- `roles.toAdd` - Array of Roles to Add
- `roles.toRemove` - Array of Roles to Remove

Response:

Success (200):

```
{
  "message": string
}
```

- `message` - Message that roles were updated on user id

Error:

```
{
  "error": string
}
```

- `error` - Error message if there was an issue

Status Codes:

- `201` - OK
- `400` - Bad request, something invalid/missing in the payload
- `401` - Username or password does not match

Group Add

Adds a group to the groups table in the database

URI: `/axapi/group`

Method: `POST`

Headers:

- `Content-Type: application/json`

Request Payload (JSON):

```
{
  "groupName": string,
  "roles": Array<String>
}
```

- `groupName` - [required] Name for the group
- `roles` - Array of roles to assign group in group-roles table

Response:

Success (200):

```
{
  "message": string
}
```

- `message` - Message indicating if group was added

Error:

```
{
  "error": string
}
```

- `error` - Error message if there was an issue

Status Codes:

- `201` - OK
- `400` - Bad request, something invalid/missing in the payload
- `401` - Username or password does not match

Group Roles Modify

Modifies a group's assigned roles in the group_roles table

URI: `/axapi/group`

Method: `PUT`

Headers:

- `Content-Type: application/json`

Request Payload (JSON):

```
{
  "groupId": int,
  "roles": {
    "toAdd": Array<String>,
    "toRemove": Array<String>
  }
}
```

- `groupId` - [required] ID for the Group
- `roles.toAdd` - Array of Roles to Add
- `roles.toRemove` - Array of Roles to Remove

Response:

Success (200):

```
{
  "message": string
}
```

- `message` - Message that roles were updated on group id

Error:

```
{
  "error": string
}
```

- `error` - Error message if there was an issue

Status Codes:

- `201` - OK
- `400` - Bad request, something invalid/missing in the payload
- `401` - Username or password does not match

Cluster Configuration

Cluster Configuration

Cluster Control

- GET Returns the cluster configuration
- POST Adds a system to the cluster
- OPTIONS Returns the allowable HTTP methods for configuration file control

System Information

Returns the list of cluster members

URI: `/axapi/cluster`

Method: `GET`

Request Payload:

```
populate=""
```

Response:

Success (200):

```
{
  "members": ["name": string,
  "address": string]
}
```

Status Codes:

- `200` - OK

System Services

System Services

Service Collection (Read Only)

Get Monitored Services

Returns the list of monitored services

URI: `/axapi/service`

Method: `GET`

Request Payload:

```
populate=""
```

Response:

Success (200):

```
{["service": string,]}
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided

Get Service Details

Returns the service details for the named service

URI: `/axapi/service/<name>`

Method: `GET`

Request Payload:

```
populate=""
```

Response:

Success (200):

```
{"service": string,"status": string}
```

Status Codes:

- `200` - - OK, command submitted (poll required)
- `400` - Invalid field value or *service* specified
- `401` - Missing/invalid token provided

Control Service

Starts or stops the named service

URI: `/axapi/service/<name>/control` Starts or stops the named service

Method: `POST`

Request Payload:

```
{"service": string,"control": string}
```

- [required] Name of the service
- [required] “start” or “stop”

Response:

Success (200):

```
{"service": string}
```

Status Codes:

- `200` - - OK
- `400` - Invalid field value or *service* specified

- `401` - Missing/invalid token provided
- `409` - Conflict or busy

Services Functions

Services utilize SQLite to store information regarding the various services. SQLite connections can be created through the SQL software. Messages received from the server will contain JSON content.

Services Collection

- GET Returns the list of all datapoints corresponding to timestamps
- PUT Creates a new entry for the database
- DELETE Clears Database
- POST Edits a service
- GET Gets a service’s status
- POST Creates new service file
- DELETE Deletes service file

Get Services

Returns data from database relating to services

URI: `/axapi/services`

Method: `GET`

Request Payload:

```
populate=""
```

Response:

Success (200):

```
{ data :[service,host,notes,]}
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided
- `500` - Error

Add a new service to database

Adds a new service to the database

URI: `/axapi/services`

Method: `PUT`

Request Payload:

```
{
  "service": string,
  "host": string,
  "notes": string
}
```

- Specifies the service name, requiredSpecifies host of service, requiredAdds notes to entry, optional

Response:

Success (200):

```
""
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided
- `500` - Error

Delete a service based on service name

Deletes a service based on service name

URI: `/axapi/services`

Method: `DELETE`

Request Payload:

```
{"service": string}
```

- Required, specifies service to delete

Response:

Success (200):

```
""
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided
- `500` - Error

Changes a service

Edits a service based on service name

URI: `/axapi/changeService`

Method: `POST`

Request Payload:

```
{ data:[service,command]}
```

Response:

Success (200):

```
""
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided
- `415` - Invalid service/command
- `500` - Error

Check status of a service

Gets status of a service based on service name

URI: `/axapi/serviceStatus/<service>`

Method: `GET`

Request Payload:

```
service=""
```

Response:

Success (200):

```
{"message": message}Where message is the status of the service
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided
- `500` - Error

Create service file

Creates a new service file using a template

URI: `/axapi/services/<filename>`

Method: `POST`

Request Payload:

```
filename=""
```

Response:

Success (200):

```
""
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided
- `500` - Error

Delete service file

Deletes a service file based on service name

URI: `/axapi/services/<filename>`

Method: `DELETE`

Request Payload:

```
filename=""
```

Response:

Success (200):

```
""
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided
- `500` - Error

Log Functions

JournalCTL and SystemCTL return valuable information that can be monitored and displayed. This data is converted from ASCII/Regular Text to Json.

Log Collection

- GET Returns the list of all log-able files

Get Logs

Gets list of loggable files

URI: `/axapi/loglist`

Method: `GET`

Request Payload:

```
populate=""
```

Response:

Success (200):

```
Loglist
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token
- `500` - Error

Extractor Functions

Similar to the config files, extractors manage a select group of files that all have to do with extracts

Extractor Collection

- GET Returns all data for an extractor based on filename
- POST Edits an extractor based on filename

Fetch File Data

Retrieves file data

URI: `/axapi/extractors/<filename>`

Method: `GET`

Request Payload:

```
filename=""
```

Response:

Success (200):

```
FileData
```

Status Codes:

- `200` - - OK, file retrieved
- `401` - Missing/invalid token provided
- `500` - Error

Edit File Data

Rewrites/creates new file based on data

URI: `/axapi/extractors/<filename>`

Method: `POST`

Request Payload:

```
filename=""
```

Response:

Success (200):

```
""
```

Status Codes:

- `200` - - OK, file edited
- `401` - Missing/invalid token provided
- `500` - Error

System Status

System Status

Get System Information

Returns the general system information

URI: `/axapi/system/information`

Method: `GET`

Request Payload:

```
populate=""
```

Response:

Success (200):

```
{
  "name": string,
  "identifier": string,
  "time": number,
  "management_containers": Boolean,
  "software": {
  "<em>packagename</em>": {
  "version": string,
  "build": string,
  "details": string
}
}
}
```

Status Codes:

- `200` - OK

License Information

Returns the system license status

URI: `/axapi/system/license`

Method: `GET`

Request Payload:

```
populate=""
```

Response:

Success (200):

```
{
  "valid": boolean,
  "maxBitRate": number,
  "expirationEpoch": number,
  "expirationTime": string,
  "features": {
  "capture": boolean,
  "replay": boolean,
  "compression": boolean,
  "encryption": boolean
}
}
```

Status Codes:

- `200` - OK

System Utilization – NOT YET IMPLEMENTED

Returns the system utilization information

URI: `/axapi/system/utilization`

Method: `GET`

Request Payload:

```
populate=""
```

Response:

Success (200):

```
{
  "memory": {
  "mem_total": string,
  "mem_free": string,
  "swap_total": string,
  "swap_free": string,
  "out_of_memory": boolean
},
  "filesystems": {
   ["&lt;name&gt;": string,]
},
  "cpuinfo": {
  "usr": string,
  "sys": string,
  "iowait": string,
  "irq": string,
  "idle": string
}
}
```

- `200` - - OK
- `400` - Invalid field value or *service* specified
- `401` - Missing/invalid token provided

Statistics

Statistics

Not yet defined

PacketXpress Configuration

PacketXpress Configuration

Configuration File Control

- GET Returns the contents of a configuration file
- POST Sets the contents of a configuration file
- OPTIONS Returns the allowable HTTP methods for configuration file control

URI	**/axapi/configuration_file/<filename>**	**Returns the contents of a specified configuration file, one of:** • **px.ini** • **site.ini** • **pxhealth.ini** • **ntservice.ini** • **01FilterMacros.ntpl** • **02AxellioDefault.ntpl**
Method	GET	
Headers	Content-Type: application/json authorization: "Bearer <token>"	
Payload (query string)		

Response	{
	"contents": string
	}
		Response for 200 status code
		The file contents

Response Status Codes	200 201 400 401	OK, file data in response OK, file does not yet exist, no file data in response Invalid file specified Missing/invalid token provided

Set Configuration File

URI	**/axapi/configuration_file/<filename>**	**Sets the contents of a specified configuration file, one of:** • **px.ini** • **site.ini** • **pxhealth.ini** • **ntservice.ini** • **01FilterMacros.ntpl** • **02AxellioDefault.ntpl**
Method	POST	
Headers	Content-Type: application/json authorization: "Bearer <token>"	

Payload (JSON)	{
	“contents”: string
	}
		The updated file contents

Response		
Response Status Codes	200 400 401 415	OK, file updated Invalid file specified Missing/invalid token provided Invalid payload

Encryption Get Status

Read the current encryption status

URI: `/axapi/encryption/status` Read the current encryption status

Method: `GET`

Request Payload: None

Response:

Status Codes:

- `200` - OK
- `401`

Encryption Unlock

Unlock the system

URI: `/axapi/encryption/status`

Method: `POST`

Request Payload:

```
{"state": "unlocked","password": string}
```

- The system encryption password

Response:

Status Codes:

- `200` - - OK, system is unlocked
- `400` - Incorrect password provided
- `401` - Missing/invalid token provided
- `415` - Invalid state provided

Encryption Change Password

Changes the system encryption password

URI: `/axapi/encryption/password`

Method: `POST`

Request Payload:

```
{
  "current password": string,
  "new password": string
}
```

- The existing password
- The desired new password

Response:

Status Codes:

- `200` - - OK, password updated
- `400` - Incorrect password provided
- `401` - Missing/invalid token provided

PCAP Extract

pcap Extract

Extract Collection

- GET Returns the list of all extracts
- POST Creates a new extract
- PUT N/A
- Delete Removes an extract
- OPTIONS Returns the allowable HTTP methods for extracts

Get list of extracts

Returns the list of known extracts

URI: `/axapi/extract`

Method: `GET`

Request Payload: None

Response:

Success (200):

```
{[{"guid": string,"time": string,"status": string},]}
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided

Create Extract With Parameters

Creates an extract with the fields provided in the payload

URI: `/axapi/extract`

Method: `POST`

Request Payload:

```
{
  "start_time": string,
  "end_time": string,
  "filter": string,
  "stream_ids": string,
  "comment": string
}
```

- The start time - RequiredThe end time - RequiredBPF filterStream IDs to extractComment is stored with the extract

Response:

Success (200):

```
{"guid": string}
```

Status Codes:

- `201` - - OK, command submitted (poll required)
- `400` - Invalid field specified
- `401` - Missing/invalid token provided
- `415` - Invalid payload

Get Extract by start time, end time, and username

Gets specified extract info from database

URI: `/axapi/guidextract/<start_time>/<end_time>/<user>`

Method: `DELETE`

Request Payload:

```
Start_time=""End_time=""User=""
```

Response:

Success (200):

```
{[StartTimeEndTimeFilterCommentUserGUID]}
```

Status Codes:

- `200` - - OK
- `400` - Invalid field specified
- `401` - Missing/invalid token provided

Get Extract by GUID

Gets specified extract info from database

URI: `/axapi/guidextract/<guid>`

Method: `DELETE`

Request Payload:

```
guid=""
```

Response:

Success (200):

```
{[StartTimeEndTimeFilterCommentUserGUID]}
```

Status Codes:

- `200` - - OK
- `400` - Invalid field or *guid* specified

- `401` - Missing/invalid token provided

Delete Extract

Deletes the specified extract

URI: `/axapi/extract/<guid>`

Method: `DELETE`

Request Payload:

```
populate=""
```

Response:

Success (200):

```
{"guid": string}
```

Status Codes:

- `200` - - OK
- `400` - Invalid field or *guid* specified
- `401` - Missing/invalid token provided

Get Extract Results

Gets the results of the specified extract

URI: `/axapi/extract/<guid>/output`

Method: `GET`

Request Payload:

```
populate=""
```

Response:

Success (200):

```
Raw Data
```

Status Codes:

- `200` - - OK, file transfer
- `400` - Invalid field or *guid* specified
- `401` - Missing/invalid token provided
- `425` - Task not complete
- `418` - No output available

PCAP

PCAP

PCAP file

- GET Returns a new PCAP as specified; creates an extract object

Get PCAP file

Returns a PCAP file

URI: `/axapi/pcap`

Method: `GET`

Headers:

- `authorization: "Bearer <token>"`

Request Payload:

```
ctid=&lt;string&gt;&amp;

start_time=&lt;time&gt;&amp;end_time=&lt;time&gt;&amp;filter=&lt;filter&gt;&amp;stream_ids=&lt;stream_i
        &amp;servers=&lt;server_list&gt;
```

- Optional client task ID to be recorded in the taskOptional sub-second epoch time for end of extractOptional sub-second epoch time for end of extractOptional Berkeley Packet Filter or community IDOptional stream IDs to extract (default “0-11”)Optional address of archive systemOptional use multi-extract; list of cluster systems, comma-separated

Response:

Success (200):

```
X-Task-GUID
```

- The unique task identifier for the PCAP extraction

Status Codes:

- `200` - - OK, file transfer
- `401` - Missing/invalid token provided
- `418` - Extraction did not provide any data (no output available)
- `500` - An error occurred while extracting

Network Flows

- GET Returns a JSON network flow for the specified request; creates a task object

Get a network flow

Returns a PCAP file

URI: `/axapi/netflow`

Method: `GET`

Headers:

- `authorization: "Bearer <token>"`

Request Payload:

```
ctid=&lt;string&gt;&amp;

start_time=&lt;time&gt;&amp;end_time=&lt;time&gt;&amp;filter=&lt;filter&gt;
```

- Optional client task ID to be recorded in the taskOptional sub-second epoch time for end of extractOptional sub-second epoch time for end of extractOptional Berkeley Packet Filter or community ID

Response:

Success (200):

```
X-Task-GUID
```

- The unique task identifier for the flow extraction

Status Codes:

- `200` - - OK, file transfer
- `401` - Missing/invalid token provided
- `418` - Extraction did not provide any data (no output available)
- `500` - An error occurred while extracting

Network Flows Stream

- GET Request network flows in an SSE stream; creates a task object

Get a SSE stream of network flows

Returns a SSE stream of network flows

URI: `/axapi/netflowstream`

Method: `GET`

Headers:

- `authorization: "Bearer <token>"`

Request Payload:

```
ctid=&lt;string&gt;&amp;

start_time=&lt;time&gt;&amp;end_time=&lt;time&gt;&amp;filter=&lt;filter&gt;&amp;lines_per_event=&lt;int
```

- Optional client task ID to be recorded in the taskOptional sub-second epoch time for end of extractOptional sub-second epoch time for end of extractOptional Berkeley Packet Filter or community IDOptional count of lines to put in an array for each SSE

Response:

Success (200):

```
X-Task-GUID
```

- The unique task identifier for the flow extraction

Status Codes:

- `200` - - OK, file transfer
- `401` - Missing/invalid token provided
- `418` - Extraction did not provide any data (no output available)
- `500` - An error occurred while extracting

SigMF Collection

- GET Returns a new SigMF file as specified; creates an extract object

Get a SIGMF file

Returns a SIGMF file

URI: `/axapi/sigmf`

Method: `GET`

Headers:

- `authorization: "Bearer <token>"`

Request Payload:

```
start_time=&lt;time&gt;&amp;end_time=&lt;time&gt;&amp;filter=&lt;filter&gt;&amp;stream_ids=&lt;stream_i
```

- Optional sub-second epoch time for end of extractOptional sub-second epoch time for end of extractOptional Berkeley Packet FilterOptional stream IDs to extract (default “0-11”)Optional passing of Json text (time/freq) to extract and uses as a filter

Response:

Success (200):

```
Raw Data
```

Status Codes:

- `200` - - OK, file transfer

- `401` - Missing/invalid token provided
- `418` - Extraction did not provide any data (no output available)
- `500` - An error occurred while extracting

SQLite Functions

SQLite connections can be created through the SQL software. Messages received from the server will contain JSON content.

SQLite Collection

- GET Returns the list of all datapoints corresponding to timestamps
- POST Clears Database

Get Datapoints between start_time and 60 seconds after

Returns data from database timestamped between start_time 60 seconds after start time

URI: `/axapi/minute/<start_time>`

Method: `GET`

Request Payload:

```
populate=""
```

Response:

Success (200):

```
{
   data :[time,
  bytes,
  streamId,
  packets,
  packetRate,
  bitRate]
}
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided
- `500` - Error

Get datapoints between start_time and 3600 seconds after

Returns data from database timestamped between start_time and 3600 seconds after start_time

URI: `/axapi/hour/<start_time>`

Method: `GET`

Request Payload:

```
populate=""
```

Response:

Success (200):

```
{
   data :[time,
  bytes,
  streamId,
  packets,
  packetRate,
  bitRate]
}
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided

- `500` - Error

Get datapoints between start_time and 86400 seconds after

Returns data from database timestamped between start_time and 86400 seconds after start_time

URI: `/axapi/day/<start_time>`

Method: `GET`

Request Payload:

```
populate=""
```

Response:

Success (200):

```
{
   data :[time,
  bytes,
  streamId,
  packets,
  packetRate,
  bitRate]
}
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided
- `500` - Error

Clear Database

Clears the database

URI: `/axapi/clear`

Method: `POST`

Request Payload:

```
populate=""
```

Response:

Success (200):

```
“”
```

Status Codes:

- `200` - - OK, database cleared
- `401` - Missing/invalid token provided
- `500` - Error

Analytics

The pxapi provides some access to the Axellio Analytics engines.

Analytics Collection

- GET Returns a process ID for a corresponding VNC display.
- POST Receives screen size data to adjust VNC display accordingly.

Get process ID for VNC display

Returns a process id for corresponding VNC display.

URI: `/axapi/analytics`

Method: `GET`

Request Payload:

```
populate=""
```

Response:

Success (200):

```
{ pid: "pid"}
```

Status Codes:

- `200` - - OK
- `500` - Error

Post dimensions of window for VNC display to adjust accordingly ***WIP***

Returns dimensions of display to adjust the VNC display accordingly.

URI: `/axapi/analytics/size`

Method: `GET`

Request Payload:

```
populate=""
```

Response:

Success (200):

```
None
```

Status Codes:

- `200` - - OK
- `401` - Missing/invalid token provided
- `415` - Invalid data entries
- `500` - Error

Reports

Conversation Counts

Request conversation counts

URI: `/axapi/analytics/reports/conversation_count`

Method: `GET`

Request Payload:

```
Date=&lt;date&gt;
```

- Optional date

Response:

Success (200):

```
{"address": integer, &lt;…&gt;}
```

Status Codes:

- `200` - - OK, command submitted (poll required)
- `400` - Invalid field or *guid* specified
- `401` - Missing/invalid token provided

Documents

Methods that relate to documentation.

Get Documents

Request list of documents in documentation directory

URI: `/axapi/documents`

Method: `GET`

Request Payload:

```
N/A
```

Response:

Success (200):

```
{"Documents": Object}
```

Status Codes:

- `200` - - OK, documents retrieved
- `401` - Missing/invalid token provided

Get Document

Request specific documentation file

URI: `/axapi/documents/<filename>`

Method: `GET`

Request Payload:

```
N/A
```

Response:

Success (200):

```
{File: File}
```

Status Codes:

- `200` - - OK, document retrieved
- `401` - Missing/invalid token provided

Filesystem Functions

Methods that relate to filesystem access.

List All Streams

Request list of all stream directories

URI: `/axapi/fs/streams`

Method: `GET`

Request Payload:

```
N/A
```

Response:

Success (200):

```
{"stream": [
```

Error (200):

```
{
  "name": string,
  "permissions": integer,
  "user": string,
  "group": string,
  "size": integer,
  "date": string,
  "type": string,
```

Error (200):

```
}, {…}]}
```

Status Codes:

- `200` - - OK, documents retrieved
- `401` - Missing/invalid token provided
- `500` - Internal

List Specific Stream Information

Request information of specific stream directory

URI: `/axapi/fs/streams?stream=<stream>`

Method: `GET`

Request Payload:

```
stream=&amp;lt;stream&amp;gt;
```

Response:

Success (200):

```
{
  "stream": {
  "name": string,
  "permissions": integer,
  "user": string,
  "group": string,
  "size": integer,
  "date": string,
  "type": string,
```

Error (200):

```
"entries": integer}}
```

Status Codes:

- `200` - - OK, documents retrieved
- `401` - Missing/invalid token provided
- `404` - Stream not found
- `500` - Internal server error

List files in stream

Request files in a stream directory

URI: `/axapi/fs/files?stream=<stream>`

Method: `GET`

Request Payload:

```
stream=&lt;stream&gt;
```

Response:

Success (200):

```
{
  "files": [{
  "name": string,
  "permissions": integer,
  "user": string,
  "group": string,
  "size": integer,
  "date": string,
  "time_start": integer,
  "time_end": integer,
  "epoch": integer,
  "seqnum": integer,
  "type": string
},
    {
  …
}
```

Error (200):

```
]}
```

Status Codes:

- `200` - - OK, documents retrieved
- `401` - Missing/invalid token provided
- `404` - Stream not found
- `500` - Internal server error

Get specific file information

Request file information

URI: `/axapi/fs/file?stream=<stream>&file=<filename>`

Method: `GET`

Request Payload:

```
stream=&lt;stream&gt;file=&lt;filename&gt;
```

Response:

Success (200):

```
{
  "files": {
  "name": string,
  "permissions": integer,
  "user": string,
  "group": string,
  "size": integer,
  "date": string,
  "time_start": integer,
  "time_end": integer,
  "epoch": integer,
  "seqnum": integer,
  "type": string
}
```

Error (200):

```
]}
```

Status Codes:

- `200` - - OK, documents retrieved
- `401` - Missing/invalid token provided
- `404` - File not found
- `500` - Internal server error

Delete file

Delete specified file in a stream directory

URI: `/axapi/fs/file?stream=<stream>&file=<filename>`

Method: `DELETE`

Request Payload:

```
stream=&lt;stream&gt;file=&lt;filename&gt;
```

Response:

Success (200):

```
{
  "status": string,
  "stream": string,
  "file": string,
  "message": string
}
```

- Status of responseStream dir related to deletionFile that was deletedMessage denoting action, "File deleted successfully"

Status Codes:

- `200` - - OK, documents retrieved
- `401` - Missing/invalid token provided
- `404` - File not found
- `500` - Internal server error

Delete Stream

Delete a stream directory

URI: `/axapi/fs/streams?stream=<stream>`

Method: `DELETE`

Request Payload:

```
stream=&lt;stream&gt;
```

Response:

Success (200):

```
{
  "status": string,
  "stream": string,
  "message": string
}
```

- Status of responseStream dir that was deletedMessage denoting action, “File deleted successfully”

Status Codes:

- `200` - - OK, documents retrieved
- `401` - Missing/invalid token provided
- `404` - Stream not found
- `500` - Internal server error

Plugin Control

The Plugin Control API provides a generic interface for managing plugins in the Axellio system. Currently supports the EncryptedSlicer plugin for controlling which encrypted protocols get sliced.

List Plugins

Returns the list of all registered plugins.

URI: `/axapi/plugins`

Method: `GET`

Response:

Success (200):

```
{
  "plugins": [
    {
      "name": "EncryptedSlicer",
      "description": "Controls which encrypted protocols get sliced"
    }
  ]
}
```

Status Codes:

- `200` - OK
- `401` - Missing/invalid token provided

Get Plugin Configuration

Returns the current configuration for a specific plugin.

URI: `/axapi/plugins/<plugin_name>/config`

Method: `GET`

Query Parameters:

- `stream` - Stream ID (integer) or `all` for all streams. Default: `all`

Response:

Success (200):

```
{
  "streamId": "0",
  "protocols": {
    "TLS_APP_DATA": false,
    "SSH_ENCRYPTED": true,
    "QUIC": true
  },
  "hasPendingChanges": false
}
```

Status Codes:

- `200` - OK
- `400` - Invalid stream parameter
- `401` - Missing/invalid token provided
- `404` - Plugin not found
- `500` - CLI command failed

Update Plugin Configuration

Updates the configuration for a specific plugin using a declarative state format.

URI: `/axapi/plugins/<plugin_name>/config`

Method: `PATCH`

Query Parameters:

- `stream` - Stream ID (integer) or `all` for all streams. Default: `all`

Request Payload:

```
{
  "protocols": {
    "TLS_APP_DATA": true,
    "QUIC": false
  }
}
```

The `protocols` object maps protocol names to boolean values:

- `true` - Enable the protocol
- `false` - Disable the protocol

Only protocols included in the request are modified; others remain unchanged.

Response:

Success (200):

```
{
  "success": true,
  "message": "Configuration updated: enabled TLS_APP_DATA; disabled QUIC",
  "stream": "0",
  "updated": {
    "TLS_APP_DATA": true,
    "QUIC": false
  }
}
```

Status Codes:

- `200` - OK
- `400` - Invalid stream parameter, missing protocols field, or protocols not an object
- `401` - Missing/invalid token provided
- `404` - Plugin not found
- `500` - CLI command failed

List Protocols

Returns the list of available protocols for the plugin (EncryptedSlicer-specific).

URI: `/axapi/plugins/<plugin_name>/protocols`

Method: `GET`

Response:

Success (200):

```
{
  "categories": {
    "Encrypted (sliced)": [
      "TLS_APP_DATA", "SSH_ENCRYPTED", "QUIC_0RTT", "QUIC",
      "WIREGUARD", "IPSEC_ESP", "DTLS_APP_DATA", "SMB_ENCRYPTED", "SRTP"
    ],
    "Encrypted (kept full)": [
      "TLS_HANDSHAKE", "TLS_PRE_LOCKIN", "SSH_HANDSHAKE",
      "QUIC_HANDSHAKE", "DTLS_HANDSHAKE"
    ],
    "Application (TCP)": [
      "HTTP", "HTTPS", "DNS_TCP", "SMTP", "IMAP", "POP3", "FTP", "TELNET",
      "RDP", "SMB", "MYSQL", "POSTGRES", "REDIS", "MONGODB", "LDAP", "KERBEROS"
    ],
    "Application (UDP)": [
      "DNS_UDP", "NTP", "DHCP", "SNMP", "SYSLOG", "TFTP", "SIP", "RTP",
      "STUN", "MDNS", "LLMNR", "NETBIOS"
    ],
    "Network": ["ICMP", "ICMPV6", "ARP", "IGMP"],
    "Catch-all": ["TCP_CONTROL", "TCP_OTHER", "UDP_OTHER", "OTHER", "FAST_TRACKED"]
  },
  "all_protocols": ["TLS_HANDSHAKE", "TLS_APP_DATA", "..."]
}
```

Status Codes:

- `200` - OK
- `401` - Missing/invalid token provided
- `404` - Plugin not found
- `500` - CLI command failed

Examples

List all plugins

```
curl -H "Authorization: Bearer <token>" \
  https://server/axapi/plugins
```

Get EncryptedSlicer config for stream 0

```
curl -H "Authorization: Bearer <token>" \
  https://server/axapi/plugins/EncryptedSlicer/config?stream=0
```

Update protocol configuration (declarative)

```
curl -X PATCH \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d '{"protocols": {"TLS_APP_DATA": true, "QUIC": false}}' \
  https://server/axapi/plugins/EncryptedSlicer/config?stream=0
```

Enable multiple protocols on all streams

```
curl -X PATCH \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d '{"protocols": {"TLS_APP_DATA": true, "SSH_ENCRYPTED": true, "QUIC": true}}' \
  https://server/axapi/plugins/EncryptedSlicer/config?stream=all
```

Disable all sliceable protocols on stream 0

```
curl -X PATCH \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d '{"protocols": {"TLS_APP_DATA": false, "SSH_ENCRYPTED": false, "QUIC": false}}' \
  https://server/axapi/plugins/EncryptedSlicer/config?stream=0
```